

Liquibase Balance Sheet & Domain Lifecycle Analyzer

Technical Architecture Design Document for IntelliJ IDEA Plugin Development

Target Platform: IntelliJ SDK / JetBrains Marketplace

Interface Architecture: JCEF (Java Chromium Embedded Framework)

Analysis Engines: IntelliJ PSI (Program Structure Interface) & Liquibase Core API

Date: July 2026

1. Executive Summary & Core Objective

In modern enterprise software engineering, maintaining absolute synchronization between application source code (Domain Entities), database migration scripts (Liquibase Changelogs), and the live production/development database schemas is a continuous challenge. Developers frequently execute direct Data Definition Language (DDL) and Data Manipulation Language (DML) modifications directly on live databases for rapid debugging, leading to **schema drift**. This drift silently invalidates the state managed by Liquibase migration files.

The goal of this plugin is to construct a native IntelliJ IDEA Tool Window capable of automatically executing a three-way architectural reconciliation between the local Liquibase XML changelogs, the live target database schema, and the application's domain-level source files (Java/Kotlin entities). Powered by an interactive JCEF front-end UI, it provides a comprehensive **Schema Balance Sheet** that detects drift, parses the semantic layout of code functionalities, and exposes a smart bi-directional wizard to heal anomalies in one click.

2. Foundational Architecture & The 3 Sources of Truth

To accurately compute database balance sheets and calculate untracked schema modifications, the plugin continuously cross-references and evaluates three decoupled representations of data architecture:

Source of Truth	Technical Target Component	Extraction Framework / Strategy
1. Expected State	Local XML Changelog Files	Parsed locally via Liquibase Core API or mapped directly into syntax structures via IntelliJ XML PSI trees.
2. Recorded History	DATABASECHANGELOG Table	Connected via JDBC; extracts metadata regarding executed changeset IDs, authors, and MD5 checksum signatures.
3. Actual Reality	Target DB Database Catalog	Live connection querying systemic schema metadata via Liquibase Snapshot and Diff Engine infrastructure.

Reconciliation Formula

The balance engine checks: Expected State (XML) $\equiv$ Recorded State (History Table) $\equiv$ Actual State (Live Catalog) .

Any structural or data divergence breaks this identity and triggers an immediate drift compilation alert.

3. UI Architecture via JCEF

To deliver a highly responsive, animated, and clear data visualization layout, the front-end uses the **Java Chromium Embedded Framework (JCEF)** embedded natively inside an IntelliJ Tool Window. This layout adapts dynamically to the current IDE canvas theme (e.g., Darcula, Light, or New UI Dark Themes).

- **The Split Balance Dashboard:** Displays a dynamic T-chart layout. The left column outlines the **Expected State (Changelog Elements)** using clean teal and blue node vectors. The right column visualizes the **Actual State (Database Topology)**.
- **Drift Alerts & KPI Tiles:** Top-aligned status cards showing real-time metrics: *Drift Score (%)*, *Bypassed DDL Events*, and *Bypassed DML Anomalies*. These elements glow dynamically in amber or crimson depending on severity thresholds.
- **Interactive Flow Nodes:** Connecting vectors bridge XML elements directly to DB tables and source fields. Clicking or hovering over an alert node surfaces a custom HTML5 tooltip containing raw audited query payloads, timestamps, and active user attribution.

4. The Core Reconciliation Logic

4.1 Changeset Metrics Strategy

The calculation of migration file anomalies evaluates changesets along the following matrix:

- **Pending Changesets:** Declared inside the master XML files but fully absent from the database tracking logs.
- **Applied Changesets:** Safely matched across local XML resources and database logs with identical checksums.
- **Orphaned/Deleted Changesets:** Logged inside the database tracking table but completely excised from the local codebase.
- **Checksum Mismatches:** Present across both contexts, but modified locally subsequent to database deployment.

4.2 Programmatic Drift & Bypass DDL Tracking

The plugin avoids invoking command-line binaries by running the Liquibase Core API natively within the plugin's background thread pool:

```
// Programmatic invocation of Liquibase comparison engine
val liveDatabase = CommandLineUtils.getDatabase(targetJdbcUrl, dbUser, dbPassword, null)
val referenceDatabase = CommandLineUtils.getDatabase("offline:postgres?changeLogFile=src/main

val compareControl = CompareControl(CompareControl.StandardTypes.DEFAULT)
val diffResult = DiffGeneratorFactory.getInstance().compare(referenceDatabase, liveDatabase,

// Extraction of structural discrepancies
val bypassedDdlList = diffResult.unexpectedObjects
val missingDdlList = diffResult.missingObjects
```

4.3 Bypassed DML Tracking Strategy

Pure row-level data alterations bypassing Liquibase scripts cannot be mapped via database metadata snapshots. The engine isolates these modifications by inspecting:

1. Target database server audit logs or query execution history views (such as PostgreSQL's `pg_stat_statements` or cloud audit systems) filtering for direct unmapped execution.
2. Data variance calculations comparing seed records defined inside local CSV/XML data files (e.g., `<loadData>`) against the actual values existing in live reference tables.

5. Deep Domain-Level Functional Analysis

Before suggesting structural migration repairs, the plugin executes an intelligent semantic analysis of the codebase at the domain level. This ensures that direct database alterations align with the application's business architecture before code generation commences.

The Strategic Rule of Domain Analysis

Never let database migration scripting execute in a silo. A database table change implies an adjustment to the underlying domain model. The plugin must use IntelliJ's index structures to establish semantic alignment before rewriting files.

5.1 Scanning via Program Structure Interface (PSI)

Using IntelliJ's internal PSI indexes, the plugin parses class components, looking for mapping structures such as Jakarta/JPA annotations (`@Entity`, `@Table`, `@Column`). It handles two critical analysis pathways:

Pathway A: Code-First Unmapped Evolution

The engine discovers that a new property has been written into a Java or Kotlin class (e.g., `private String phoneNumber;` inside `Supplier.java`) and finds a corresponding column manually appended to the live database, but no matching Liquibase block exists. The plugin concludes that this is a valid domain model change that simply skipped migration assembly.

Pathway B: Isolated Database Structural Bypass

The engine detects an `ALTER TABLE` modification in the live catalog, but the corresponding entity class has no matching field. The wizard instantly raises a critical impact alert, noting that executing an automated Liquibase script synchronization will result in a field mismatch inside the runtime code structure.

5.2 Programmatic Code Parsing via PSI

The snippet below illustrates how the plugin safely accesses short-name data caches to locate and evaluate the target domain class without forcing an active compilation cycle:

```
fun analyzeDomainMapping(project: Project, targetTableName: String): PsiClass? {
    val shortNamesCache = PsiShortNamesCache.getInstance(project)

    // Normalize SQL snake_case to Java/Kotlin CamelCase naming structures
    val targetClassName = targetTableName.split("_").joinToString("") { it.capitalize() }

    val classesFound = shortNamesCache.getClassesByName(
        targetClassName,
        GlobalSearchScope.projectScope(project)
    )

    return classesFound.firstOrNull { psiClass ->
        psiClass.hasAnnotation("jakarta.persistence.Entity") ||
        psiClass.hasAnnotation("javax.persistence.Entity")
    }
}
```

6. Automated Reconciliation & Bi-Directional Healing

Once the three-way analysis finishes processing, the user can execute the **Smart Remediation Wizard** inside the JCEF interface to fix layout imbalances across code, tracking files, and databases simultaneously.

6.1 Automated Creation of New XML Changesets

The structural layout variances resolved within `diffResult` feed into the `DiffToChangeLog` pipeline to automatically construct semantic changesets matching standard syntax configurations:

```
val diffOutputConfig = DiffOutputConfig(false, false, false, null)
val changeLogSerializer = ChangeLogSerializerFactory.getInstance().getSerializer("xml")

// Transform the calculated structural variance directly into Liquibase configuration element
val generatedChangeSets = DiffToChangeLog(diffResult, diffOutputConfig).generateChangeSets()
appendChangeSetsToMasterFile(projectChangelogPath, generatedChangeSets, changeLogSerializer)
```

6.2 Core Synchronization Baseline

To guarantee that subsequent CI/CD orchestration layers do not execute these newly generated elements a second time on the target database, the plugin automatically markers them as deployed by initiating an isolated baseline sync action:

```
// Programmatic execution of the changeLogSync command targeting the freshly injected block 1
val liquibaseContext = Liquibase(projectChangeLogPath, ResourceAccessor(), liveDatabase)
liquibaseContext.changeLogSync(Contexts(), LabelExpression())
```

6.3 Bi-Directional Healing Options

Depending on the analysis pathway resolved, the wizard triggers one of three targeted workflows:

1. **Generate Code-First Changeset:** Appends matching Liquibase XML migration tags directly into local files to cleanly mirror the layout configurations discovered within the active JPA entities.
2. **Inject Domain Code Field:** Injects the missing Java or Kotlin private field alongside getter/setter annotations straight into the source `PsiClass` code block to perfectly mirror unmapped database schema properties.
3. **DML SQL Wrap:** Extracts the raw SQL data payload captured from historical database statements, packages it inside a custom `<sql>` changeset block, and commits a tracking row to prevent execution duplication.